

Reviewer Pack — Minimal Rank of Tropicalization over Communication Channel E...

Entry 08fb0714c272 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 10:32:29 UTC

Minimal Rank of Tropicalization over Communication Channel Entropy

Entry ID: 08fb0714c272 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 10:32:29 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Communication Complexity

Statement:

{‘sentence_1’: “For any given communication channel, the minimal rank of its tropicalized input space is upper bounded by a function of the channel’s entropy.”, ‘sentence_2’: ‘Specifically, for a communication channel with entropy H , the minimal rank of its tropicalized input space is $O(2^H)$.’, ‘sentence_3’: ‘For all instances with entropy H , if the minimal rank exceeds 2^H , then the channel requires at least an exponential number of bits to communicate.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘The relationship between tropical geometry and communication complexity is relatively unexplored, suggesting a potential new angle for proving lower bounds.’, ‘sentence_2’: ‘Tropicalization provides a bridge between discrete algebraic structures and continuous analysis, which may reveal structural insights into communication tasks.’, ‘sentence_3’: ‘Entropies often serve as fundamental measures in information theory, and bounding their relationship with tropical geometric properties could lead to novel complexity-theoretic results.’}

Taxonomy category: TROPICAL_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6d2e95992c70f12b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a communication channel with entropy H , if the measured minimal rank of its tropicalized input space exceeds 2^H for any seed, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry AND communication complexity AND channel entropy - minimal rank tropicalization communication complexity channel entropy - entropic bound on minimal rank tropical geometry communication channels

Top relevant hits considered: - [<http://arxiv.org/abs/1207.2443v2>] Tropical Teichmuller and Siegel spaces - [<http://arxiv.org/abs/1204.3875v2>] Tropicalizing vs Compactifying the Torelli morphism - [<http://arxiv.org/abs/1204.6154v2>] Local Tropicalization - [<http://arxiv.org/abs/1403.8106v1>] Recent advances on the log-rank conjecture in communication complexity - [<http://arxiv.org/abs/quant-ph/9911079v3>] Minimal Entropy of States Emerging from Noisy Quantum Channels - [<http://arxiv.org/abs/2002.00000v1>] Minimal volume entropy of simplicial complexes - [<http://arxiv.org/abs/1610.00298v4>] Khovanskii bases, higher rank valuations and tropical geometry - [<http://arxiv.org/abs/1207.1925v1>] Introduction to tropical algebraic geometry

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_channel(n):
        return [random.choice([0, 1]) for _ in range(n)]

    def tropicalize(channel):
        return max(channel)

    def entropy(channel):
        counts = {0: channel.count(0), 1: channel.count(1)}
        total = len(channel)
        if total == 0:
            return 0
        p0, p1 = counts[0] / total, counts[1] / total
        return -p0 * math.log2(p0) - p1 * math.log2(p1)

    def minimal_rank(channel):
```

```

    n = len(channel)
    rank = 0
    for i in range(n):
        if channel[i] == 1:
            rank += 1
    return rank

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    for _ in range(5): # Test with 5 random seeds per n
        channel = generate_channel(n)
        rank = minimal_rank(channel)
        H = entropy(channel)
        results.append({
            "n": n,
            "channel": channel,
            "rank": rank,
            "H": H,
            "conjecture_holds": rank <= 2 ** H
        })

instances_tested = len(results)
conjecture_holds_all = all(r["conjecture_holds"] for r in results)
counterexample = next((f"n={r['n']}, H={r['H']:.2f}, rank={r['rank']}" for r in results if not r["conjecture_holds"]))

return {
    "metric_name": "Minimal Rank",
    "metric_value": sum(r["rank"] for r in results) / instances_tested,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds_all,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) if len(sys.argv) > 1 else [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        print(f"TRIAL: {run_trial(seed)}")
        results.append(run_trial(seed))

    mean_d = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_d:.2f} std=0.00 support_fraction=1.00")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\n{n={results[0]['n']}, H={results[0]['H']:.2f}, r={results[0]['rank']}"")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c0a4fc9f.py", line 77, in <module>
    print(f"TRIAL: {run_trial(seed)}")
                      ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c0a4fc9f.py", line 50, in run_trial
    H = entropy(channel)
          ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c0a4fc9f.py", line 33, in entropy
    return -p0 * math.log2(p0) - p1 * math.log2(p1)
           ~~~~~^
ValueError: math domain error
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a math domain error before producing data that could confirm or falsify the conjecture. | next: Investigate and fix the math domain error in the test code, then rerun the test with all seeds to determine if the conjecture is supported or falsified.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11540
2	preregistration	ollama_remote	glm4:latest	0	0	5331
3	novelty	ollama_remote	glm4:latest	0	0	4449
4	novelty	ollama_remote	glm4:latest	0	0	5927
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12522
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9250
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6579
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8962
9	judge	ollama_remote	glm4:latest	0	0	8350

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 72911 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/08fb0714c272.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/08fb0714c272.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/08fb0714c272.tar.gz` (if generated)